SQL — Jointures avancées & Sous-requêtes

Semaine 3 · Jeudi · 1h · C5 — Créer une BDD | C6 — Composants d'accès aux données

Base de données : Application mobile de livraison de repas

1. Concepts clés

1.1 LEFT JOIN & RIGHT JOIN

Rappel : INNER JOIN ne retourne que les lignes qui ont une correspondance des deux côtés. LEFT et RIGHT JOIN conservent toutes les lignes d'un côté, même sans correspondance.

LEFT JOIN vs RIGHT JOIN — Application livraison

restaurants

id	nom
1	Pizza Roma
2	Sushi Bar
3	Burger King
4	Tacos House

commandes

id	restaurant_id	total
1	1	24.50
2	1	18.00
3	3	32.00
4	3	12.00

LEFT JOIN : tous les restaurants + commandes (

r.nom	c.id	c.total
Pizza Roma	1	24.50
Pizza Roma	2	18.00
Sushi Bar	NULL	NULL
Burger King	3	32.00
Burger King	4	12.00
Tacos House	NULL	NULL

← NULL : pas de commande

RIGHT JOIN : toutes les commandes + restaurant (NUL

c.id	c.total	r.nom
1	24.50	Pizza Roma
2	18.00	Pizza Roma
3	32.00	Burger King
4	12.00	Burger King

Note pratique :
RIGHT JOIN est rare. On préfère inverser l'ordre des tables et utiliser LEFT JOIN.

Type de JOIN	Comportement — Cas d'usage
INNER JOIN	Lignes avec correspondance des deux côtés uniquement. Usage : données liées garanties (commande + client).
LEFT JOIN	Toutes les lignes de la table GAUCHE + correspondances de droite (NULL si absent). Usage : 'tous les restaurants, même sans commande'.
RIGHT JOIN	Toutes les lignes de la table DROITE + correspondances de gauche. Rarement utilisé — on préfère inverser les tables et faire un LEFT JOIN.

1.2 Sous-requêtes — IN, NOT IN, EXISTS

Une sous-requête est un SELECT imbriqué dans un autre. Elle s'exécute d'abord et son résultat est utilisé par la requête principale.

Sous-requêtes — IN, NOT IN, EXISTS

IN — id existe dans la sous-requête

```
-- Clients ayant passé au moins une commande
SELECT nom FROM utilisateurs
WHERE id IN (
    SELECT DISTINCT client_id
    FROM commandes
);
```

NOT IN — id absent de la sous-requête

```
-- Clients n'ayant JAMAIS commandé
SELECT nom FROM utilisateurs
WHERE type = 'client'
AND id NOT IN (
    SELECT DISTINCT client_id
    FROM commandes
);
```

EXISTS — la sous-requête retourne au moins 1 ligne

Quand utiliser quoi ?

IN	Liste de valeurs connue ou résultat d'une SELECT renvoyant peu de lignes
NOT IN	Exclusion — attention si la sous-requête peut retourner NULL (résultat inattendu !)
EXISTS	Vérifier l'existence d'une ligne liée. Plus performant que IN sur grandes tables

⚠ **Piège NOT IN avec NULL :**
Si la sous-requête retourne une valeur NULL, NOT IN retourne toujours FALSE

⚠ Piège NOT IN + NULL : si la sous-requête retourne au moins un NULL, NOT IN retourne toujours vide. Préférer NOT EXISTS dans ce cas.

Performance : EXISTS s'arrête à la première ligne trouvée. IN charge toutes les valeurs en mémoire. Sur grandes tables, EXISTS est plus rapide.

1.3 GROUP BY + HAVING + JOIN combinés

On peut combiner JOIN, GROUP BY et HAVING dans la même requête pour produire des statistiques croisées entre plusieurs tables.

GROUP BY + HAVING + JOIN — Requêtes analytiques livraison

Table commandes (toutes lignes) → **GROUP BY** restaurant_id (agrégation) → **HAVING** SUM > 40€ (filtre groupes) → **Résultat** final (restaurants OK)

Rappel clé :
WHERE filtre les LIGNES (avant groupement) ≠ HAVING filtre les GROUPES (après agrégation)

Nb commandes + CA par restaurant

```
SELECT r.nom,
  COUNT(c.id)   AS nb_cmd,
  SUM(c.total)  AS ca
FROM commandes c
JOIN restaurants r ON r.id=c.restaurant
GROUP BY r.id, r.nom
ORDER BY ca DESC;
```

Restaurants CA > 40€ seulement

```
SELECT r.nom, SUM(c.total) AS ca
FROM commandes c
JOIN restaurants r ON r.id=c.restaurant
GROUP BY r.id, r.nom
HAVING SUM(c.total) > 40
ORDER BY ca DESC;
```

Livreurs > 2 livraisons ce mois

```
SELECT u.nom,
  COUNT(c.id) AS nb_livraisons
FROM commandes c
JOIN utilisateurs u ON u.id=c.livreur_i
WHERE c.statut = 'livré'
GROUP BY u.id, u.nom
HAVING COUNT(c.id) > 2;
```

Ordre d'écriture SQL : SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
Ordre d'exécution réel : FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
WHERE filtre avant regroupement (sur les lignes). HAVING filtre après (sur les groupes).

1.4 EXPLAIN — Analyser les performances

EXPLAIN affiche le plan d'exécution que MySQL choisit pour une requête. C'est l'outil de base pour détecter les requêtes lentes et savoir où placer des index.

EXPLAIN — Analyser le plan d'exécution d'une requête

```
EXPLAIN SELECT u.nom, COUNT(c.id) AS nb
FROM commandes c
JOIN utilisateurs u ON c.client_id = u.id
GROUP BY u.id;
```

Résultat EXPLAIN :

table	type	key	rows	Extra
commandes	ALL	NULL	1000	Using temporary
utilisateurs	ref	PRIMARY	1	Using index

→ ALL = scan complet → LENT sans index
→ ref = utilise un index → RAPIDE
→ rows = nombre de lignes examinées

INDEX — Pourquoi et comment ?

Sans index	MySQL parcourt TOUTES les lignes (full table scan) → lent sur grandes tables
Avec index	MySQL saute directement aux lignes concernées → très rapide
Créer index	CREATE INDEX idx_client ON commandes(client_id);
Quand ?	Sur les colonnes utilisées dans WHERE, JOIN ON, ORDER BY, GROUP BY

Colonne EXPLAIN	Signification
type = ALL	Scan complet de la table → très lent sur grandes tables. Indique qu'un index manque.
type = ref	Utilise un index non-unique → rapide.
type = const	Utilise la clé primaire ou un index unique → le plus rapide possible.
key	Nom de l'index utilisé. NULL = aucun index utilisé → à corriger.
rows	Estimation du nombre de lignes examinées. Plus c'est bas, mieux c'est.

```
-- Créer un index sur client_id pour accélérer les JOINs
CREATE INDEX idx_client ON commandes(client_id);

-- Créer un index composé (utile pour WHERE multi-colonnes)
CREATE INDEX idx_statut_date ON commandes(statut, created_at);

-- Vérifier les index existants
SHOW INDEX FROM commandes;
```

2. Exercices pratiques

✏ Exercice 1 — Tableau de bord livraison — Guidé (30 min)

Contexte :

Le directeur de l'application de livraison veut un tableau de bord analytique. Il a besoin de plusieurs statistiques croisées : performances par restaurant, activité des livreurs, fidélité des clients. Toutes les requêtes utilisent la base de données livraison vue depuis le début de la semaine.

Rappel schéma utilisé :

- `utilisateurs(id, nom, email, type)` — type: 'client' | 'livreur'
- `restaurants(id, nom, ville, note_moy)`
- `plats(id, restaurant_id, nom, prix, categorie)`
- `commandes(id, client_id, livreur_id, restaurant_id, statut, total, created_at)`
- `lignes_commande(id, commande_id, plat_id, quantite, prix_unit)`

- `notations(id, commande_id, note, commentaire)`

Requêtes à écrire :

1. LEFT JOIN — Tous les restaurants avec leur nombre de commandes (y compris ceux à 0 commande)
2. LEFT JOIN + GROUP BY — Pour chaque livreur, afficher son nom et le nombre de livraisons effectuées (statut = 'livré'), même s'il n'en a aucune
3. Sous-requête IN — Afficher les clients qui ont passé au moins une commande dont le total dépasse 30€
4. Sous-requête NOT IN — Afficher les restaurants qui n'ont reçu AUCUNE commande
5. GROUP BY + HAVING — Restaurants ayant reçu plus de 3 commandes ET un chiffre d'affaires total > 80€
6. JOIN 3 tables + GROUP BY — Pour chaque client, son nom et la somme totale dépensée, triée du plus gros au plus petit
7. Sous-requête EXISTS — Livreurs ayant au moins une notation > 4 (via la table notations et commandes)
8. EXPLAIN — Lancez EXPLAIN sur la requête n°6 et identifiez si un index manque. Créez-le si nécessaire.

🚀 **Bonus :**

a. Trouvez le plat le plus commandé (toutes commandes confondues) en croisant lignes_commande et plats
b. Calculez le panier moyen par ville en croisant commandes et restaurants, uniquement pour les villes avec plus de 5 commandes

✏ Exercice 2 — Analyse RH — Semi-autonome (30 min)

Contexte :

Même base de données, mais cette fois on analyse les données sous l'angle des ressources humaines de la plateforme : performances des livreurs, satisfaction client par livreur, et identification des livreurs inactifs. L'objectif est de produire des requêtes analytiques plus complexes combinant plusieurs techniques vues aujourd'hui.

Requêtes à écrire :

9. Pour chaque livreur, afficher : nom, nombre total de livraisons, montant total livré, note moyenne reçue (LEFT JOIN sur notations via commandes)
10. Identifier les livreurs 'stars' : ceux ayant plus de 2 livraisons effectuées ET une note moyenne >= 4 (GROUP BY + HAVING sur jointure 4 tables)
11. Livreurs inactifs : ceux qui n'ont AUCUNE commande avec statut 'livré' ce mois-ci (NOT EXISTS ou NOT IN avec filtre sur created_at)
12. Classer les livreurs par taux de succès : (nb commandes 'livré' / nb commandes totales) * 100, uniquement pour ceux ayant au moins 3 commandes
13. Pour chaque restaurant, afficher la note moyenne des notations laissées par les clients (via commandes → notations), avec au moins 1 notation
14. EXPLAIN sur la requête n°1 — identifier les colonnes sans index et créer les index manquants

🚀 **Bonus :**

c. Comparer les performances des livreurs entre deux périodes : ce mois vs le mois précédent (sous-requêtes corrélées)
d. Trouver les clients 'VIP' : ceux dont la somme dépensée est supérieure à la moyenne de tous les clients (sous-requête scalaire dans HAVING)